Tutorial: Alertas e Relatórios do Zabbix pelo WhatsApp

Este guia leva você do zero até o primeiro cliente recebendo alertas e relatórios no WhatsApp. Faça uma vez as etapas 1 a 3 (infraestrutura da empresa). Depois, repita a etapa 4 para cada cliente novo.

Etapa	O que é	Quando
1. Preparar o número	Chip e celular dedicados	Uma vez
2. Instalar a Evolution API	Servidor que conecta o WhatsApp	Uma vez
3. Instalar o gerador de relatórios	Script Python + cron	Uma vez
4. Cadastrar um cliente	Alertas + relatórios do cliente	Por cliente
5. Operação do dia a dia	Monitoramento, backup, atualização	Contínuo

Antes de começar

Você vai precisar de:

- **Um servidor Linux da empresa** (VM ou VPS) com 1 vCPU, 2 GB de RAM e 20 GB de disco, Debian 12, Ubuntu 22.04/24.04 ou Rocky/Alma 9. É nele que ficam a Evolution API e o gerador de relatórios.
- **Um chip de celular exclusivo** para o monitoramento e um aparelho para ler o QR code (pode ser um celular antigo).
- **Acesso de administrador (Super admin)** ao Zabbix de cada cliente.
- **Um nome DNS com HTTPS** para a Evolution (ex.: `whatsapp.suaempresa.com.br`), se algum Zabbix de cliente estiver fora da rede desse servidor. É o caso comum quando cada cliente tem o próprio Zabbix.

Os caminhos de menu do Zabbix aparecem assim: **7.x** primeiro, *(6.0 entre parênteses)*.

Etapa 1 — Preparar o número de WhatsApp

1. Ative o chip num celular e instale o **WhatsApp Business** (recomendado) ou o WhatsApp comum.
2. Configure o perfil: nome "Monitoramento - Sua Empresa", foto com o logo e uma descrição como "Número automático de alertas. Não responda.".
3. Use o número por alguns dias antes de colocar em produção: entre nos grupos, troque algumas mensagens. Números novos que disparam muitas mensagens logo de cara têm mais risco de bloqueio.
4. **Não use esse número para mais nada.**

Regras para não ter o número bloqueado:

- Envie só para grupos onde o número foi adicionado e para pessoas que salvaram o contato.
- Peça aos clientes para salvar o número e não marcá-lo como spam.
- Ajuste as ações do Zabbix para não mandar alertas de severidade baixa (Informação/Não classificada) para o WhatsApp.

Etapa 2 — Instalar a Evolution API

Todos os comandos desta etapa rodam no servidor da empresa, como root.

2.1 Instalar o Docker

```
curl -fsSL https://get.docker.com | sh
systemctl enable --now docker
```

2.2 Subir a Evolution

Copie a pasta `evolution/` do pacote para `/opt/evolution` e entre nela:

```
mkdir -p /opt/evolution
cp evolution/docker-compose.yml evolution/.env.example /opt/evolution/
cd /opt/evolution
cp .env.example .env
```

Gere duas senhas fortes:

```
openssl rand -hex 32
openssl rand -hex 16
```

Edite o `.env` (`nano .env`):

- `AUTHENTICATION_API_KEY` = a primeira senha. Esta é a **apikey**, que você vai usar no Zabbix e no script. Guarde-a num cofre de senhas.
- `POSTGRES_PASSWORD` = a segunda senha. Coloque a mesma senha também dentro de `DATABASE_CONNECTION_URI`, no lugar de `troque-esta-senha`.
- `SERVER_URL` = a URL pública, se for usar HTTPS (ex.: `https://whatsapp.suaempresa.com.br`).

Antes de subir, confira em https://github.com/EvolutionAPI/evolution-api/releases qual é a versão estável mais recente e ajuste a linha `image:` do `docker-compose.yml`. Depois:

```
docker compose up -d
docker compose ps
docker compose logs -f evolution
```

Espere aparecer que o servidor iniciou na porta 8080 e saia dos logs com Ctrl+C.

2.3 Criar a instância e conectar o WhatsApp

Crie a instância chamada `zabbix` (troque `SUA_APIKEY`):

```
curl -s -X POST http://127.0.0.1:8080/instance/create \
  -H "apikey: SUA_APIKEY" -H "Content-Type: application/json" \
  -d '{"instanceName":"zabbix","integration":"WHATSAPP-BAILEYS","qrcode":true}'
```

Para ler o QR code, abra o painel `http://IP-DO-SERVIDOR:8080/manager`. Se a porta estiver presa ao 127.0.0.1, use um túnel SSH a partir do seu computador:

```
ssh -L 8080:127.0.0.1:8080 usuario@servidor
```

Depois abra `http://localhost:8080/manager` no navegador. Entre com a apikey, abra a instância `zabbix` e clique para gerar o QR code. No celular do chip: **WhatsApp › Aparelhos conectados › Conectar um aparelho** e leia o código.

Confirme a conexão. A resposta deve conter `"state":"open"`:

```
curl -s http://127.0.0.1:8080/instance/connectionState/zabbix -H "apikey: SUA_APIKEY"
```

2.4 Enviar uma mensagem de teste

Troque pelo seu número, com 55 + DDD:

```
curl -s -X POST http://127.0.0.1:8080/message/sendText/zabbix \
  -H "apikey: SUA_APIKEY" -H "Content-Type: application/json" \
  -d '{"number":"5511999998888","text":"Teste da Evolution API"}'
```

Se a mensagem chegou no seu WhatsApp, a etapa 2 está pronta.

2.5 Publicar com HTTPS (quando o Zabbix do cliente está em outra rede)

O `docker-compose.yml` deixa a porta 8080 acessível só no próprio servidor. Para que o Zabbix de um cliente consiga enviar alertas, publique a Evolution com HTTPS e **libere apenas os IPs dos Zabbix dos clientes**. A apikey dá controle total do número, então ela nunca deve ficar exposta na internet sem restrição.

Exemplo com nginx + Let's Encrypt (Debian/Ubuntu):

```
apt install -y nginx certbot python3-certbot-nginx
```

Crie `/etc/nginx/sites-available/evolution`:

```
server {
    listen 80;
    server_name whatsapp.suaempresa.com.br;

    # Zabbix dos clientes (um allow por IP público) e a rede interna
    allow 203.0.113.10;   # Acme Ltda
    allow 198.51.100.20;  # Beta Transportes
    allow 10.0.0.0/8;
    deny all;

    client_max_body_size 20m;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Ative o site e emita o certificado:

```
ln -s /etc/nginx/sites-available/evolution /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx
certbot --nginx -d whatsapp.suaempresa.com.br
```

Para o certbot validar o domínio, a porta 80 precisa estar acessível pela internet durante a emissão. Se o `deny all` bloquear a validação, adicione temporariamente `allow all;` e remova depois.

A cada cliente novo, inclua o IP do Zabbix dele com um `allow` e rode `systemctl reload nginx`.

Etapa 3 — Instalar o gerador de relatórios

No mesmo servidor, com Python 3.9 ou mais novo (Debian 12, Ubuntu 22.04+ e Rocky 9 já vêm com ele):

```
apt install -y python3 python3-venv
cd report
sudo ./install.sh
```

No Rocky/Alma, troque o primeiro comando por `dnf install -y python3`. O instalador:

- cria o usuário de sistema `zbxreport`;
- instala o script em `/opt/zabbix-whatsapp`, com um ambiente Python isolado (venv);
- cria `/opt/zabbix-whatsapp/config.yaml` a partir do exemplo;
- cria o comando `zbxrep`, um atalho para rodar o gerador (usado no resto deste tutorial);
- cria `/etc/cron.d/zabbix-whatsapp` com os agendamentos **desativados**;
- gera um PDF de demonstração em `/var/lib/zabbix-whatsapp/relatorios/` para provar que tudo funciona.

Edite a configuração e preencha a seção `evolution` com a URL (`http://127.0.0.1:8080`, já que roda no mesmo servidor), a instância `zabbix` e a apikey:

```
nano /opt/zabbix-whatsapp/config.yaml
```

Deixe os clientes de exemplo por enquanto; você vai trocá-los na etapa 4. Em `company_name`, coloque o nome da sua empresa, que aparece no rodapé do PDF.

Confira se a Evolution está acessível pelo script:

```
zbxrep --list-groups
```

A saída deve listar os grupos de que o número participa (ou nada, se ele ainda não está em nenhum grupo), sem erro.

Etapa 4 — Cadastrar um cliente

Faça esta etapa uma vez em cada Zabbix. Se vários clientes ficam no mesmo Zabbix central, importe o tipo de mídia só uma vez e repita a partir do passo 4.3 para cada cliente.

4.1 Importar o tipo de mídia

1. **Alerts › Media types** *(Administration › Media types)* › botão **Import**.
2. Escolha `mediatype/whatsapp_evolution_mediatype.yaml` e confirme.
3. Abra o tipo **WhatsApp (Evolution API)** e preencha os parâmetros:

Parâmetro	Valor
`evolution_url`	`https://whatsapp.suaempresa.com.br`, ou `http://127.0.0.1:8080` se o Zabbix estiver no mesmo servidor
`evolution_instance`	`zabbix`
`evolution_apikey`	a apikey da etapa 2
`zabbix_url`	URL do frontend deste Zabbix, ex. `https://zabbix.acme.com.br` (vai no link de cada alerta; deixe vazio se o cliente não acessa o Zabbix)

Não altere os parâmetros que têm macros entre chaves `{...}`.

4.2 Testar o tipo de mídia

Ainda no tipo de mídia, clique em **Test**. Preencha `send_to` com o seu número (`5511999998888`), `event_source` = `0`, `event_value` = `1`, `event_nseverity` = `4`, `event_name` = `Teste`, `host_name` = `teste`, e clique em **Test**. Deve aparecer `OK` e chegar um alerta "PROBLEMA | Alta" no seu WhatsApp.

Se aparecer erro, veja a tabela "Solução de problemas" no fim deste guia.

4.3 Criar o grupo de WhatsApp do cliente

1. No celular do chip, crie um grupo "Monitoramento - Acme Ltda" com a equipe de vocês e os contatos do cliente. Ou peça para o cliente adicionar o número a um grupo existente.
2. Descubra o identificador (JID) do grupo:

```
zbxrep --list-groups
```

A saída tem uma linha por grupo, com o JID e o nome, por exemplo `120363025555555555@g.us Monitoramento - Acme Ltda`. Anote o JID; ele termina em `@g.us`.

4.4 Criar o usuário que recebe os alertas

O Zabbix só envia alerta para usuários que têm **permissão de leitura** nos hosts do problema. Este é o erro mais comum.

1. **Users › User groups** *(Administration › User groups)* › **Create user group**:
   - Nome: `WhatsApp - Acme`
   - Aba **Host permissions** *(Permissions)*: adicione os grupos de hosts do cliente com **Read**.
2. **Users › Users** *(Administration › Users)* › **Create user**:
   - Username: `whatsapp-acme`, Groups: `WhatsApp - Acme`, senha forte qualquer (ninguém vai logar com ele).
   - Aba **Media** › **Add**: Type `WhatsApp (Evolution API)`, Send to `120363025555555555@g.us`. Para mandar também para um técnico, acrescente o número separado por vírgula: `120363025555555555@g.us, 5511999998888`. Em *Use if severity*, marque Média, Alta e Desastre (sugestão).
   - Aba **Permissions**: Role `User role`.

Para o plantão da sua empresa, crie também um usuário `whatsapp-plantao` com leitura em **todos** os grupos e o número/grupo de plantão. Coloque a tag `cliente` nos hosts (ex.: `cliente` = `Acme Ltda`) para que cada alerta mostre de qual cliente ele é.

4.5 Criar a ação

Alerts › Actions › Trigger actions *(Configuration › Actions › Trigger actions)* › **Create action**:

- **Action**: Name `WhatsApp - Acme`. Conditions: `Host group equals <grupos do cliente>` e, se quiser, `Trigger severity is greater than or equals Average`.
- **Operations**:
  - Operations › Add: *Send to users* `whatsapp-acme`, *Send only to* `WhatsApp (Evolution API)`.
  - Recovery operations › Add: *Notify all involved*.
  - Update operations › Add: *Notify all involved* (avisa quando alguém reconhece ou comenta o problema).

Salve. Para testar de verdade, provoque um problema controlado num host do cliente (pare um serviço monitorado ou use uma trigger de teste) e confirme que chegam o "PROBLEMA" e depois o "RESOLVIDO".

4.6 Liberar o acesso do relatório à API do Zabbix

1. Crie um user group `Relatórios` com **Read** nos grupos de hosts do cliente e um usuário `relatorios` nele, com Role `User role`.
2. **Users › API tokens** *(Administration › General › API tokens)* › **Create API token**: User `relatorios`, Name `whatsapp-relatorios`, desmarque *Set expiration date*. Copie o token gerado; ele aparece uma única vez.
3. Garanta que o servidor da empresa alcança o frontend do Zabbix do cliente (HTTPS 443). Se o Zabbix do cliente não é público, use VPN ou libere o IP do seu servidor no firewall do cliente.

No Zabbix 6.0 sem API tokens disponíveis, use `zabbix_user` e `zabbix_password` no lugar do token.

4.7 Adicionar o cliente no config.yaml

```
nano /opt/zabbix-whatsapp/config.yaml
```

Na lista `clients:`, adicione o bloco abaixo e apague os clientes de exemplo:

```
  - name: "Acme Ltda"
    zabbix_url: https://zabbix.acme.com.br
    zabbix_token: "COLE_O_TOKEN_AQUI"
    send_to:
      - "120363025555555555@g.us"
    reports: [weekly, monthly]
```

Opções por cliente:

Opção	Para quê	Padrão
`host_groups`	Só considerar estes grupos (obrigatório num Zabbix central com vários clientes)	todos
`reports`	Quais relatórios o cliente recebe: `daily`, `weekly`, `monthly`	o de `defaults`
`min_severity`	Severidade mínima contada no relatório (2 = Atenção)	2
`availability_severity`	A partir de qual severidade conta como indisponível (4 = Alta)	4
`attach_pdf`	Enviar o PDF além do texto	true
`verify_ssl`	`false` para certificado autoassinado	true
`timezone`	Fuso do cliente	America/Sao_Paulo

O YAML é sensível à indentação: use espaços, nunca TAB, e mantenha o alinhamento dos exemplos.

4.8 Testar e ativar o relatório

Primeiro, gere sem enviar e confira o texto e o PDF:

```
zbxrep --period weekly --client "Acme Ltda" --dry-run
```

O caminho do PDF aparece no fim da saída. Copie para o seu computador (`scp`) e abra. Quando estiver bom, envie de verdade uma vez:

```
zbxrep --period weekly --client "Acme Ltda"
```

Ative o agendamento removendo o `#` do início das linhas desejadas:

```
nano /etc/cron.d/zabbix-whatsapp
```

Com os horários padrão, os relatórios saem:

- **Diário:** todo dia às 07:00, cobrindo o dia anterior.
- **Semanal:** segunda às 07:10, cobrindo segunda a domingo da semana anterior.
- **Mensal:** dia 1 às 07:20, cobrindo o mês anterior.

Os horários seguem o fuso do servidor; confira com `timedatectl`. Se não for `America/Sao_Paulo`, ajuste com `timedatectl set-timezone America/Sao_Paulo`. Cada execução só envia para os clientes que têm aquele período em `reports`.

Para um relatório avulso (por exemplo, pedido de um cliente sobre um incidente), indique o intervalo:

```
zbxrep --client "Acme Ltda" --start 2026-10-01 --end 2026-10-31
```

Checklist de cliente novo

- ☐ Número do monitoramento adicionado ao grupo do cliente
- ☐ IP do Zabbix do cliente liberado no nginx (`allow`)
- ☐ Tipo de mídia importado, parâmetros preenchidos, botão Test OK
- ☐ User group com Read nos hosts + usuário `whatsapp-<cliente>` com a mídia
- ☐ Ação criada com operações de problema, recuperação e atualização
- ☐ Tag `cliente` nos hosts (para o grupo de plantão)
- ☐ Usuário `relatorios` + API token criados
- ☐ Bloco do cliente no `config.yaml`
- ☐ `--dry-run` conferido e um envio real feito
- ☐ Cliente orientado a salvar o número do monitoramento

Etapa 5 — Operação do dia a dia

Vigiar a própria integração

Se o WhatsApp desconectar, os alertas param de sair sem ninguém perceber. Para evitar isso:

1. No Zabbix da empresa: **Data collection › Templates** *(Configuration › Templates)* › **Import** › `mediatype/template_evolution_monitor.yaml`.
2. Crie um host `evolution-api` (pode ser o próprio servidor) e vincule o template **WhatsApp Evolution API by HTTP**.
3. Na aba **Macros** do host, preencha `{$EVOLUTION.URL}`, `{$EVOLUTION.INSTANCE}` e `{$EVOLUTION.APIKEY}`.
4. Crie uma ação que envie os alertas desse host por **e-mail, Telegram ou SMS**, nunca pelo WhatsApp.

O template avisa quando a conexão sai do estado `open` ou quando a Evolution para de responder.

Para monitorar os relatórios, consulte o log em `/var/log/zabbix-whatsapp/relatorios.log`. Toda linha `ERROR` indica um cliente que não recebeu.

Quando o WhatsApp desconecta

Abra o `/manager`, gere um novo QR code na instância `zabbix` e leia com o celular do chip. Mantenha esse celular ligado e com internet de vez em quando: se ficar mais de 14 dias desconectado, o WhatsApp encerra as sessões de aparelhos conectados.

Atualizar a Evolution

```
cd /opt/evolution
docker compose pull
docker compose up -d
```

Para mudar de versão, troque a tag `image:` no `docker-compose.yml` antes do `pull`. Leia as notas da versão no GitHub antes de atualizar.

Backup

O que precisa de backup:

- `/opt/evolution/.env`
- `/opt/zabbix-whatsapp/config.yaml`
- os volumes do Docker (`evolution_instances` e `postgres_data`), que guardam a sessão do WhatsApp. Sem eles, você só precisa ler o QR code de novo.

Rotação de log

Crie `/etc/logrotate.d/zabbix-whatsapp`:

```
/var/log/zabbix-whatsapp/*.log {
    monthly
    rotate 12
    compress
    missingok
    notifempty
    su zbxreport zbxreport
}
```

Atualizar o gerador de relatórios

Com uma versão nova da pasta `report/`, rode o instalador de novo. Ele preserva o `config.yaml` e o cron:

```
cd report
sudo ./install.sh
```

Personalização

- **Texto dos alertas:** edite `mediatype/whatsapp_evolution.js`. Rode `node mediatype/test_webhook.js` para testar e `node mediatype/build_mediatype.js` para gerar o YAML novo; depois, importe de novo no Zabbix, marcando para atualizar o existente. Se preferir os templates de mensagem do próprio Zabbix, mude o parâmetro `use_default_format` para `false`.
- **Rodapé e nome no PDF:** `company_name` no `config.yaml`.
- **Conteúdo do PDF:** função `build_pdf` em `zabbix_whatsapp_report.py`.

Solução de problemas

Sintoma	Causa e solução
Test do tipo de mídia: `HTTP 401`	apikey errada em `evolution_apikey`
`HTTP 404`	nome da instância errado em `evolution_instance`, ou URL errada
`HTTP 400` com `exists: false`	número sem WhatsApp ou sem o 55 na frente
`Couldn't connect` / timeout	o Zabbix não alcança a Evolution: DNS, firewall, `allow` do nginx
Test OK, mas alertas reais não chegam	usuário sem permissão Read nos hosts, mídia com severidade desmarcada, ou ação com condição errada. Veja em **Reports › Action log**
Alerta chega no número, mas não no grupo	o número saiu do grupo ou o JID mudou; rode `--list-groups`
Mensagens param de chegar de repente	WhatsApp desconectado: confira o `connectionState` e leia o QR code
Relatório: `Not authorised`	token errado, expirado, ou usuário sem Read nos grupos
Relatório: aviso "grupos não encontrados"	nome em `host_groups` diferente do Zabbix (maiúsculas e barras contam)
Relatório: erro de certificado SSL	`verify_ssl: false` no cliente (só para certificado autoassinado)
Relatório não saiu pelo cron	veja `/var/log/zabbix-whatsapp/relatorios.log`; confirme que a linha do cron está sem `#`

Para mais detalhes de um erro no relatório, repita o comando com `-v`.